

Explain Longest DAG Path

Longest Path in a Directed Acyclic Graph (DAG)

This problem is very similar to **Shortest Path in a DAG**, but instead of finding the **minimum distance**, we want the **maximum distance**.
The most important thing is:

We can find the longest path efficiently **only because the graph is a DAG**.

A DAG has **no cycles**, so we can process vertices in **topological order**.

1. What is the problem asking?

You are given:

- V vertices from 0 to $V-1$
- Directed weighted edges
- A source vertex `src`

Each edge is:

```
[u, v, w]
```

meaning:

```
u ----w----> v
```

You need to find:

```
Longest distance from src to every vertex
```

If a vertex cannot be reached from `src`, return:

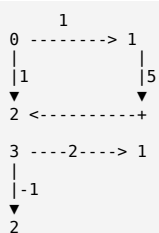
```
C++
INT_MIN
```

Example 1

```
V = 4
src = 0

Edges:
0 → 1 (1)
0 → 2 (1)
1 → 2 (5)
3 → 1 (2)
3 → 2 (-1)
```

Graph:



Starting from 0:

Vertex 1

```
0 → 1
distance = 1
```

Vertex 2

There are two possible paths:

```
0 → 2 = 1
```

and

```
0 → 1 → 2 = 1 + 5 = 6
```

We want the **longest**:

```
distance[2] = 6
```

Vertex 3 cannot be reached from 0.

Therefore:

```
[0, 1, 6, INT_MIN]
```

2. Why can't we use normal Dijkstra?

Dijkstra is designed for **shortest paths** and relies on the idea that once the shortest distance to a node is finalized, it won't improve later.

For longest paths, this logic does not work in a general graph.

Also consider a positive cycle:

```

0 → 1 → 2
  ↑   |
  +---+
  +5

```

You could keep going around the cycle:

```
0 → 1 → 2 → 1 → 2 → ...
```

and the path length would become infinitely large.

But in a **DAG**, cycles do not exist.

So every path eventually ends.

This is why we can solve it using:

Topological Sort + Relaxation

3. Main intuition

First, get a **topological order**.

For example:

```
0 → 1 → 2
```

A valid topological order is:

```
0, 1, 2
```

The important property is:

```
u comes before v
```

for every edge:

```
u → v
```

Now when we process **u**, we already know the longest distance to **u**.

Then we use that information to update its neighbors.

For an edge:

```
u ----w----> v
```

If:

```
dist[u] = longest distance from src to u
```

then one possible path to **v** is:

```
dist[u] + w
```

So:

```
C++
dist[v] = max(dist[v], dist[u] + w);
```

This is the core formula.

4. Important initialization

For shortest path, we usually do:

```
C++
dist[i] = INF;
```

For longest path, we need:

```
C++
dist[i] = INT_MIN;
```

Why?

Because initially we haven't found any path to that node.

Then:

```
C++
dist[src] = 0;
```

So:

```
dist = [INT_MIN, INT_MIN, ..., INT_MIN]
dist[src] = 0
```

5. The relaxation step

Suppose:

```
dist[u] = 10
u ----4----> v
```

Then going through *u* gives:

```
10 + 4 = 14
```

So:

```
C++
dist[v] = max(dist[v], 14);
```

In code:

```
C++
dist[v] = max(dist[v], dist[u] + weight);
```

But there is an important condition:

```
C++
if (dist[u] != INT_MIN)
```

Why?

Suppose *u* is unreachable:

```
C++
dist[u] = INT_MIN
```

Then we should not try:

```
C++
```

```
INT_MIN + weight
```

because there is no valid path from `src` to `u`.

So:

```
C++
if (dist[u] != INT_MIN) {
    dist[v] = max(dist[v], dist[u] + weight);
}
```

Approach 1: DFS Topological Sort + Relaxation

This is one common solution.

Step 1: Create adjacency list

For every edge:

```
[u, v, w]
```

store:

```
C++
adj[u].push_back({v, w});
```

Example:

```
0 → 1 (1)
0 → 2 (1)
1 → 2 (5)
```

becomes:

```
0 : (1,1), (2,1)
1 : (2,5)
2 :
```

Step 2: Find topological order using DFS

The DFS logic:

```
DFS(node)
├── visit all neighbors
└── push node into stack
```

Notice:

We push the node **after visiting all its neighbors**.

Example:

```
0 → 1 → 2
```

DFS execution:

```
DFS(0)
  DFS(1)
    DFS(2)
      push 2
    push 1
  push 0
```

Stack:

```
top
0
1
2
```

When popped:

```
0, 1, 2
```

which is a valid topological order.

Complete DFS Solution

```
C++
class Solution {
private:
    void topoSort(int node,
                 vector<vector<pair<int, int>>>& adj,
                 vector<int>& visited,
                 stack<int>& st) {

        visited[node] = 1;

        for (auto [next, weight] : adj[node]) {
            if (!visited[next]) {
                topoSort(next, adj, visited, st);
            }
        }

        st.push(node);
    }

public:
    vector<int> longestPath(int V, vector<vector<int>>>& edges, int src) {

        // Step 1: Create adjacency list
        vector<vector<pair<int, int>>> adj(V);

        for (auto& edge : edges) {
            int u = edge[0];
            int v = edge[1];
            int w = edge[2];

            adj[u].push_back({v, w});
        }

        // Step 2: Find topological order
        vector<int> visited(V, 0);
        stack<int> st;

        for (int i = 0; i < V; i++) {
            if (!visited[i]) {
                topoSort(i, adj, visited, st);
            }
        }

        // Step 3: Initialize distance
        vector<int> dist(V, INT_MIN);

        dist[src] = 0;

        // Step 4: Process nodes in topological order
        while (!st.empty()) {

            int node = st.top();
            st.pop();

            // Skip unreachable nodes
            if (dist[node] == INT_MIN) {
                continue;
            }

            for (auto [next, weight] : adj[node]) {

                dist[next] = max(
                    dist[next],
                    dist[node] + weight
                );
            }
        }

        return dist;
    }
};
```

Let's dry run Example 1

```
V = 4
src = 0

Edges:
0 → 1 (1)
0 → 2 (1)
1 → 2 (5)
3 → 1 (2)
3 → 2 (-1)
```

One possible topological order:

```
3 → 0 → 1 → 2
```

Initial distance:

```
dist = [0, -∞, -∞, -∞]
```

Here:

```
-∞ = INT_MIN
```

Process 3

```
dist[3] = -∞
```

So 3 is unreachable from source 0.

We skip it:

```
C++
if (dist[3] == INT_MIN)
    continue;
```

Distance remains:

```
[0, -∞, -∞, -∞]
```

Process 0

```
dist[0] = 0
```

Edges:

```
0 → 1 (1)
0 → 2 (1)
```

Update:

```
dist[1] = max(-∞, 0 + 1)
         = 1
```

```
dist[2] = max(-∞, 0 + 1)
         = 1
```

Now:

```
[0, 1, 1, -∞]
```

Process 1

Edge:

```
1 → 2 (5)
```

Current:

```
dist[1] = 1
```

New path:

```
1 + 5 = 6
```

Update:

```
dist[2] = max(1, 6)
         = 6
```

Now:

```
[0, 1, 6, -∞]
```

Process 2

No outgoing edges.

Final:

```
[0, 1, 6, INT_MIN]
```

Approach 2: Kahn's Algorithm (BFS Topological Sort)

Instead of DFS, we can find the topological order using **indegree**.

Idea

For every node, calculate:

```
indegree[node]
```

Indegree means:

How many edges are coming into this node?

Example:

```
0 → 1 → 2
```

Indegree:

```
0 = 0
1 = 1
2 = 1
```

We first put all nodes having:

```
indegree = 0
```

into a queue.

Then:

```
remove node
process its edges
decrease indegree of neighbors

if indegree becomes 0:
    push neighbor
```

This generates a topological order.

Kahn's Algorithm Code

```
C++
class Solution {
public:
    vector<int> longestPath(int V, vector<vector<int>>> edges, int src) {
        vector<vector<pair<int, int>>> adj(V);
        vector<int> indegree(V, 0);

        // Build graph
        for (auto& edge : edges) {
            int u = edge[0];
            int v = edge[1];
            int w = edge[2];

            adj[u].push_back({v, w});
            indegree[v]++;
        }

        // Queue for topological sort
        queue<int> q;

        for (int i = 0; i < V; i++) {
            if (indegree[i] == 0) {
                q.push(i);
            }
        }

        // Distance array
        vector<int> dist(V, INT_MIN);
        dist[src] = 0;

        // Topological processing
        while (!q.empty()) {
            int node = q.front();
            q.pop();

            // Relax edges only if reachable
            if (dist[node] != INT_MIN) {
                for (auto [next, weight] : adj[node]) {
                    dist[next] = max(
                        dist[next],
                        dist[node] + weight
                    );
                }
            }
        }
    }
};
```

```

    });
}
}

// Update indegrees
for (auto [next, weight] : adj[node]) {
    indegree[next]--;
    if (indegree[next] == 0) {
        q.push(next);
    }
}
}

return dist;
}
};

```

A slightly cleaner Kahn's approach

I personally think this version is easier to understand because we first create the complete topological order.

```

C++
class Solution {
public:
    vector<int> longestPath(int V, vector<vector<int>>& edges, int src) {

        vector<vector<pair<int, int>>> adj(V);
        vector<int> indegree(V, 0);

        // Build graph
        for (auto& edge : edges) {
            int u = edge[0];
            int v = edge[1];
            int w = edge[2];

            adj[u].push_back({v, w});
            indegree[v]++;
        }

        // Find topological order
        queue<int> q;

        for (int i = 0; i < V; i++) {
            if (indegree[i] == 0) {
                q.push(i);
            }
        }

        vector<int> topo;
        while (!q.empty()) {
            int node = q.front();
            q.pop();

            topo.push_back(node);

            for (auto [next, weight] : adj[node]) {
                indegree[next]--;
                if (indegree[next] == 0) {
                    q.push(next);
                }
            }
        }

        // Longest distances
        vector<int> dist(V, INT_MIN);
        dist[src] = 0;

        // Relax edges in topological order
        for (int node : topo) {
            if (dist[node] == INT_MIN) {
                continue;
            }

            for (auto [next, weight] : adj[node]) {
                dist[next] = max(
                    dist[next],
                    dist[node] + weight
                );
            }
        }

        return dist;
    }
};

```

Approach 3: Recursion + Memoization

There is also a DP way to think about this problem.

Define:


```
dp(node) = longest distance from src to node
```

But calculating from `src` directly is slightly awkward because a node can have multiple parents.

Instead, reverse the thinking:

```
dp(node) = longest path starting from node
```

Then:

```
dp(node) = max(weight + dp(next))
```

However, this calculates the longest path **starting from a node**, not specifically the distance from `src` to every vertex.

To get the exact required answer for every vertex, we could process recursively from the source and update distances, but topological sorting is much cleaner and more standard.

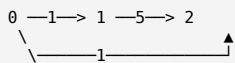
So for this specific problem:

Topological Sort + Relaxation is the preferred approach.

The DP interpretation

This is actually a Dynamic Programming problem on a DAG.

Consider:



For node 2:

```
dist[2]
=
maximum of all possible ways to reach 2
```

Possible transitions:

```
dist[2] = dist[0] + 1
```

or:

```
dist[2] = dist[1] + 5
```

Therefore:

```
C++
dist[v] = max(dist[v], dist[u] + weight);
```

This is exactly like DP:

```
previous state + transition value
```

The topological order guarantees that:

Before calculating a node, all possible previous nodes have already been processed.

Shortest Path vs Longest Path in DAG

Feature	Shortest Path	Longest Path
Initial value	INF	INT_MIN
Source	0	0
Relaxation	min()	max()
Formula	dist[v] = min(dist[v], dist[u] + w)	dist[v] = max(dist[v], dist[u] + w)
Unreachable	INF	INT_MIN
Works with negative weights?	Yes, in DAG	Yes, in DAG

The biggest difference is just this:

Shortest path

```
C++
dist[v] = min(dist[v], dist[u] + weight);
```

Longest path

```
C++
dist[v] = max(dist[v], dist[u] + weight);
```

Very important: Negative weights are allowed

For example:

```
0 --(-2)--> 1 --(-3)--> 2
```

Then:

```
dist[0] = 0
dist[1] = -2
dist[2] = -5
```

This is still valid.

Why?

Because the graph is a DAG.

There is no cycle that can be repeatedly traversed.

Time and Space Complexity

Let:

```
V = number of vertices
E = number of edges
```

DFS Topological Sort

```
Topological sort: O(V + E)
Relaxation:      O(V + E)
```

Total:

```
O(V + E)
```

Space:

```
Adjacency list = O(V + E)
Visited        = O(V)
Stack          = O(V)
Distance       = O(V)
```

Overall:

```
O(V + E)
```

The easiest way to remember the solution

Just remember this template:

```
1. Build adjacency list
2. Find topological order
3. Initialize:
   dist[src] = 0
   everything else = INT_MIN
4. Process nodes in topological order
5. For every edge:
   u -> v (weight)
   if u is reachable:
       dist[v] =
           max(dist[v],
              dist[u] + weight)
```

Core logic

```
C++
for (int node : topo) {
    if (dist[node] == INT_MIN) {
        continue;
    }
    for (auto [next, weight] : adj[node]) {
        dist[next] = max(
            dist[next],
            dist[node] + weight
        );
    }
}
```

So the entire problem can be summarized as:

Topological Sort tells us the correct order to process nodes, and DP relaxation keeps the maximum distance found so far.

If you want, the next useful step would be to take **Example 2** and do a complete visual dry run showing the **topological order, every relaxation, and exactly how [INF, 0, 9, 10, 4] is formed.**

ChatGPT can make mistakes. Check important info.